
Speaker Recognition With Fun Application

Project Description

In this project, we explored the effectiveness of a speaker recognition system. Using our computer's built-in microphone, we collected sound data and extracted features using two different methods: a fine-tuned model and a homegrown CNN. Once the system was implemented, we evaluated its performance in a real-world application to determine how well it would hold up under semi-practical conditions.

The system has two main modes: enrollment and identification. By running a specific command in the terminal, users can specify which mode they would like to use, as well as which feature extraction method to apply.

The enrollment mode prompts the user to enter their name and records eight four-second audio samples of them speaking. The identification mode runs continuously, recording four-second intervals and deriving similarity scores for each recording. If the person is identified as someone in the database, it is considered a successful match, and the application is activated.

Model Training / Fine-tuning

As stated above, our system uses two models to extract features from .wav files. Both models were either trained or fine-tuned on the VoxCeleb1 dataset. This dataset contains audio samples of different actors from movies, stored as .wav files. A subset of these files was then loaded and resampled to 16 kHz before being preprocessed for the respective models.

Fine-Tuned Feature Extraction

The fine-tuned feature extraction method consists of a few preprocessing steps before the audio is passed through the model. The audio is first resampled to 16 kHz if it is not already in that format. It is then trimmed to a four-second window. This processed audio is then passed through the feature extractor for the WavLM model. WavLM is a transformer-based model that outputs a 256-dimensional feature vector.

Homegrown Feature Extraction

The homegrown feature extraction method also consists of a few preprocessing steps before the data can be passed through the model. The audio, similar to the fine-tuned method, is first resampled to 16 kHz if it is not already in that format. The audio is then converted into a log-Mel spectrogram.

The log-Mel spectrogram is then passed into the model. Our model consists of a small, three-layer convolutional neural network. Once the input has passed through the CNN layers, it is represented as a set of 256-dimensional tokens. These tokens are then passed through a transformer encoder, where the final output token is used as the feature vector. This feature vector has a length of 256.

Log-Mel Spectrogram

A log-Mel spectrogram is a type of spectrogram designed to mimic human hearing. This is important because it enables machine learning models to better learn distinctions between different voices. Although it is abstracted through the `wav_to_log_mel` function in python, a log-Mel spectrogram is created through multiple steps.

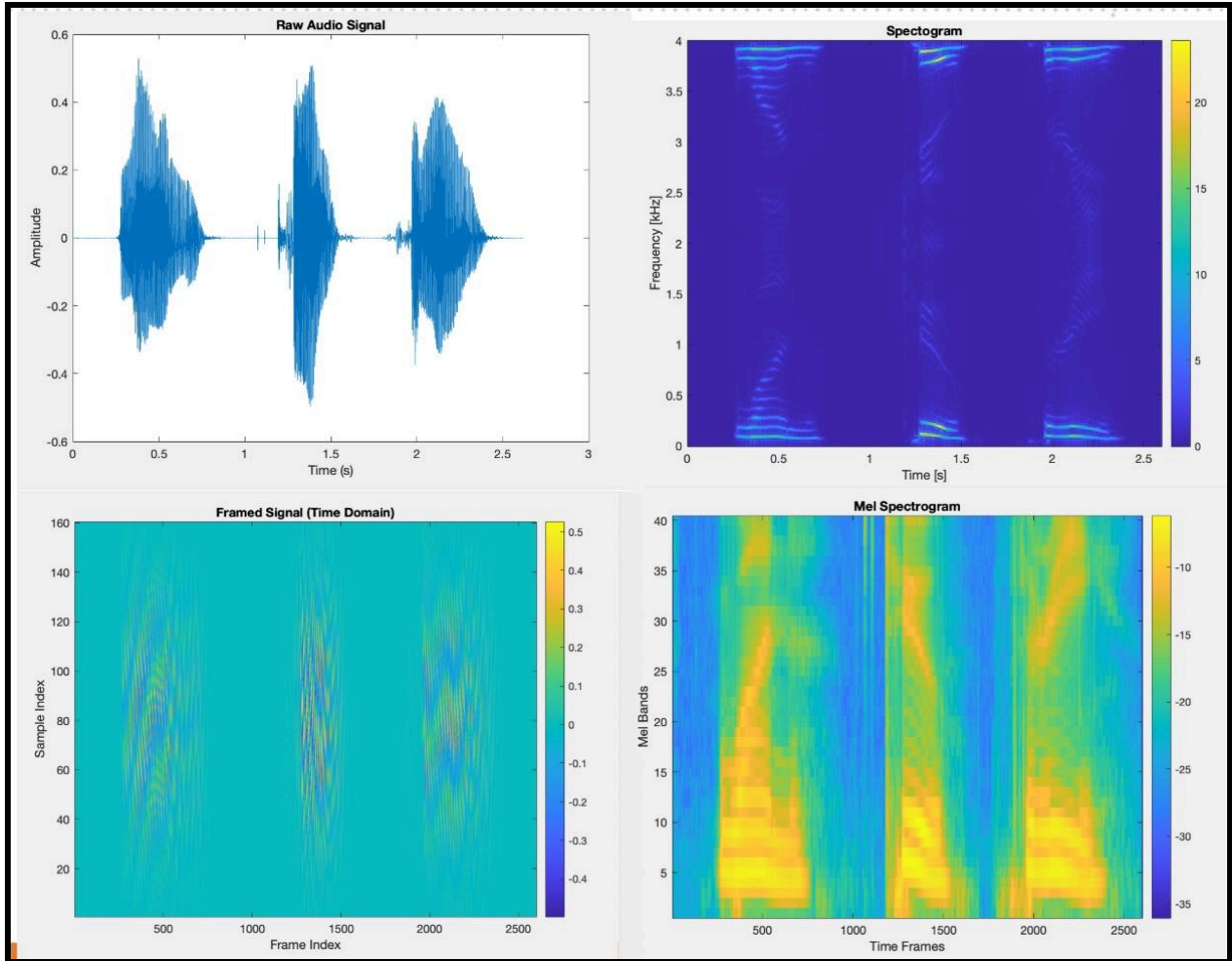


Figure 1: Top Left: The Raw Audio Signal. Bottom Left: Framed Signal. Top Right: The initial Spectrogram. Bottom Right: The resulting log-Mel Spectrogram

The first step is to frame the signal into a matrix of short segments. Framing is necessary to capture the time-varying characteristics of the signal. Because audio signals are inherently time-dependent, not applying framing would result in the loss of important information. The framed signal is shown on the bottom left of Figure 1.

The second step is to apply the Fourier transform to each framed signal. This converts the signal from the time domain into the frequency domain, providing a significantly more informative representation of the signal. This results in the normal spectrogram, shown in the top right of Figure 1. There are also two important factors to consider when taking the Fourier transform. First, it is important to use the Fast Fourier Transform (FFT) of the signal, as it is significantly less computationally expensive. By reducing the number of multiplications and relying more on additions, the algorithm can be computed much more efficiently on computers. Second, when taking the Fourier transform of a signal, it is important to properly address aliasing. As seen in the spectrogram in Figure 1, the signal appears to repeat itself at higher frequencies due to aliasing. To combat this, the signal should be sampled and framed at no

more than half the sampling frequency. This helps ensure that only the true signal components are captured without overlap from aliased frequencies.

The third step is to convert the spectrogram into the Mel scale. This is done by sending our spectrogram through the Mel Filter Banks, which are a set of overlapping triangular filters. These filters have different sizes making sure the signal represents human hearing. Finally, the logarithmic transform is applied to further develop the spectrogram. The final log-Mel spectrogram is shown on the bottom right of Figure 1.

Database

An sqlite database is used to store the users and their embeddings for each of the models. To store the embeddings an sqlite extension called sqlite-vec is used, sqlite-vec does not allow foreign keys so an intermediate table is used to link the user to their embeddings. The database schema contains three tables: the users table, the embeddings table, and the user_embeddings table. The users table stores the users and whether or not they are a target, the embeddings table stores the feature embeddings, and the user_embeddings table is the bridge between the users and embeddings table it stores which row in the embeddings table belongs to which user and which model that embedding came from.

Performance

Using cosine similarity to determine the similarity score, our system performs relatively well. As shown in Figure 2, our fine-tuned model had an EER of .128 at a threshold of .859. In addition, both the DET and ROC curve are close to the bottom left and top right corners respectively.

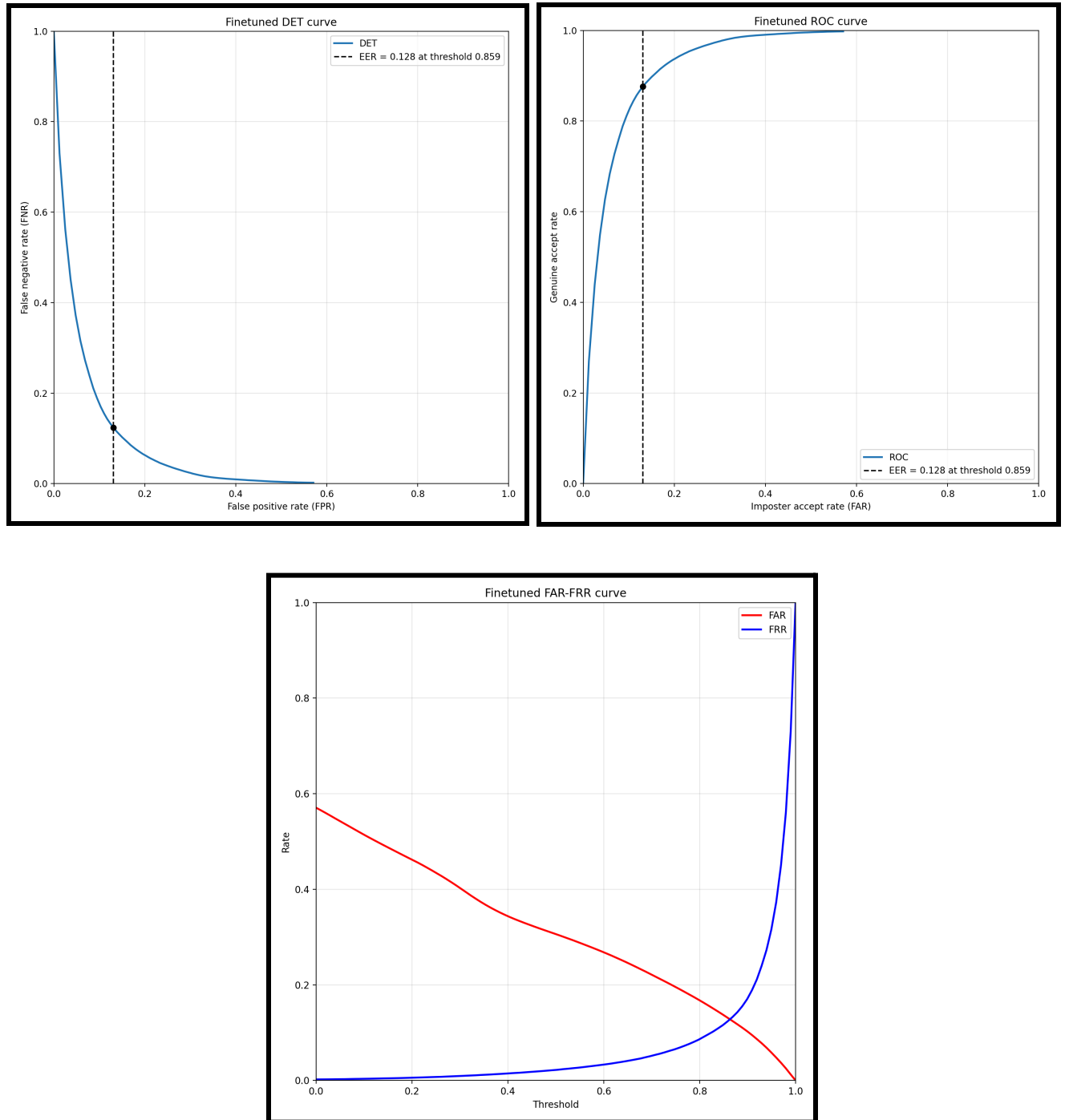


Figure 2: Top Left: Fine-tuned DET curve. Top Right: Fine-tuned ROC curve. Bottom: Fine-tuned FAR-FRR curve.

As shown in Figure 3, our homegrown system had an EER value of .152 at a threshold of .556. In addition, similar to the fine-tuned, both the DET and ROC curve are close to the bottom left and top right corners respectively.

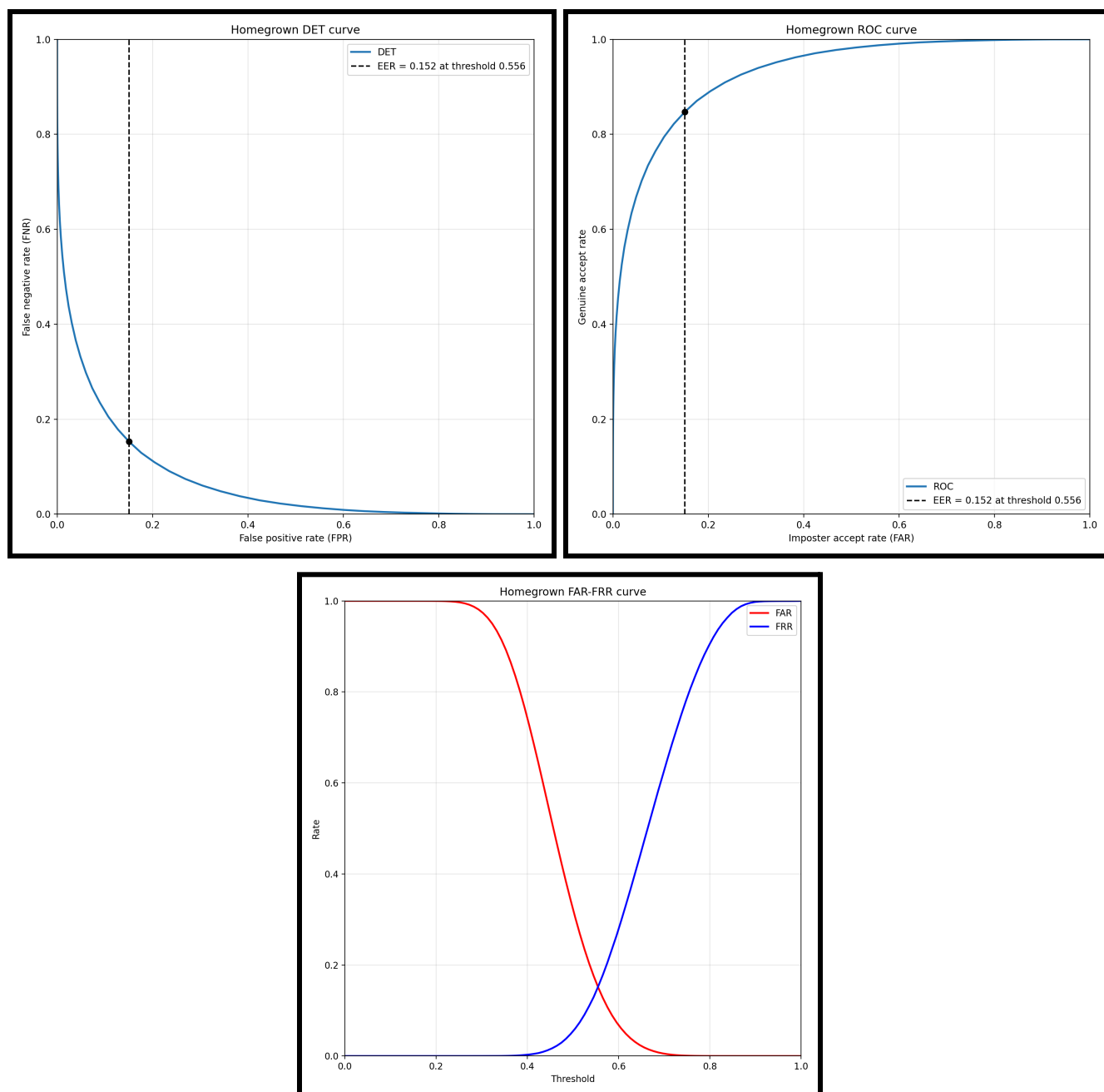


Figure 3: Top Left: Homegrown DET curve. Top Right: Homegrown ROC curve. Bottom: Homegrown FAR-FRR curve

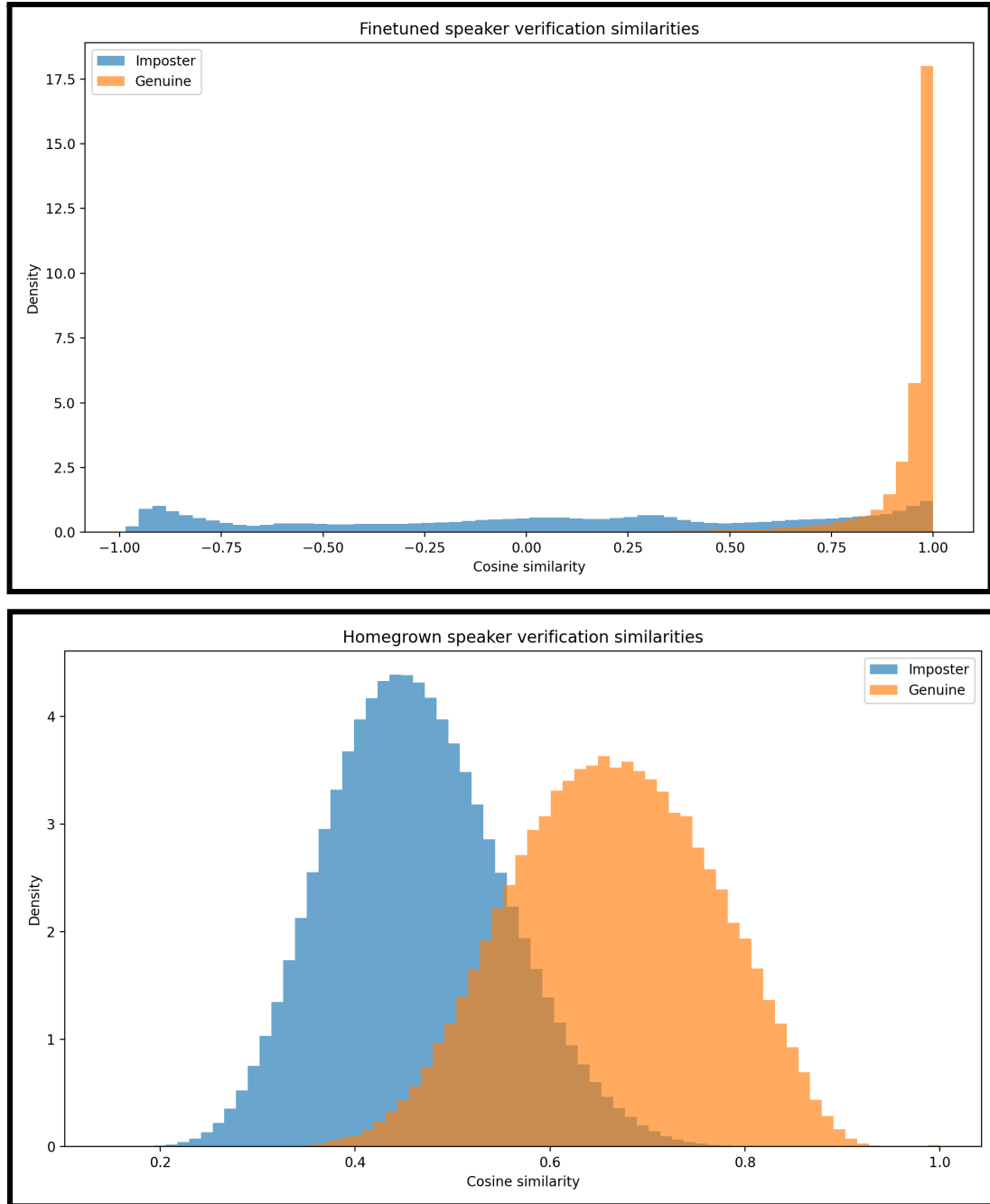


Figure 4: This image shows our fine-tuned (top) and homegrown (bottom) similarity histogram plots

As shown in Figure 4, our homegrown model appears to have much more overlap than the fine-tuned model. However the axes are very different, the x-axis for the fine-tuned model goes into the negatives while the homegrown model only goes to about zero. What this means is that the overlaps are in actuality pretty similar. The negative values from the fine-tuned model reveals that this model most likely learns richer representations since it is creating embeddings that are nearly opposite to each other indicating that these embeddings could be better.

However in practice we observed that the homegrown performed better even though the performance indicated that it was slightly worse than the fine-tuned model. In addition, we observed that our system, for our application, functioned better at a threshold of .9, for the fine-tuned, and .7, for the homegrown, due to our desire to even further reduce false acceptances. Finally, we noticed that our system, in general, had a small similarity score when comparing a female to male voice, but had a much higher similarity score when comparing a male to male voice. This reinforces that real life testing is extremely necessary for any product as real world results may be very different from controlled testing setups.

Application

To test our system in a real-world application, we integrated it into a Nerf blaster. The goal was to allow the blaster to be used only when a voice was recognized, meaning the user must be authorized before operating the device.

The hardware consisted of four main components: a Feather ESP32-S3 used as the controller, an IRLB8314PBF MOSFET used as the electrical switch, the DC motors provided in the blaster, and the 9V battery pack included with the blaster.

The circuit operates by connecting the Feather ESP32-S3 to a computer through USB. The Feather is preloaded with Arduino code that causes it to wait for information sent from the computer via USB. When the computer successfully identifies a user's voice as one stored in the database, it sends a high signal through USB to the Feather. The Feather then sets GPIO pin 9 high, which closes the MOSFET and allows voltage to reach the motors. After 5 seconds, the computer sends a low signal to turn the motors off. The system then continues monitoring for a recognized voice. The circuit diagram is shown below in Figure 5. It should also be noted that a flyback diode was used in the circuit to improve safety. Although it is included in the schematic, it is embedded within the MOSFET diagram.

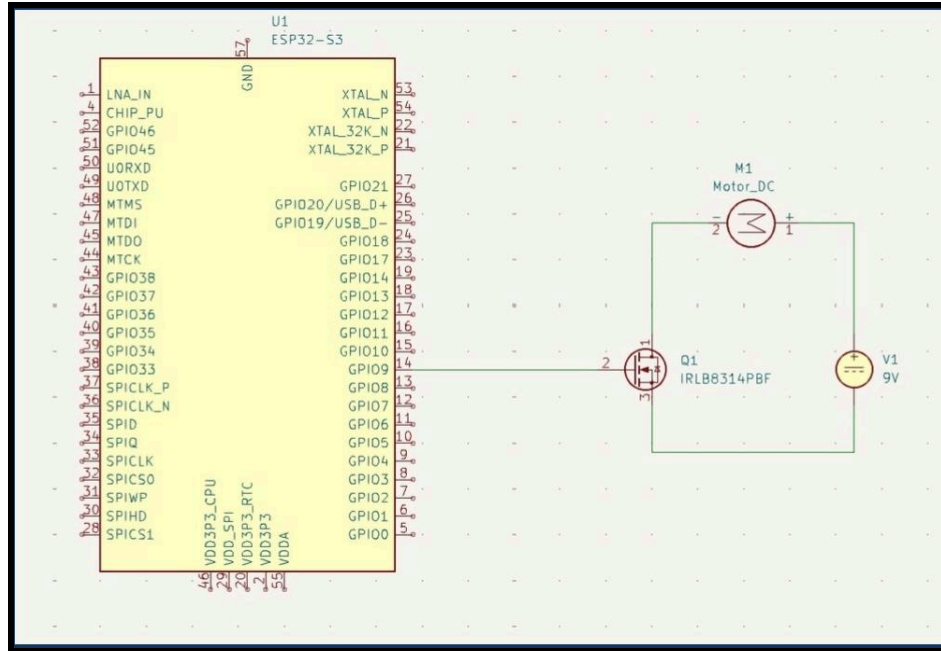


Figure 5: Circuit Diagram of the hardware in KiCad

This application proved to be both a fun and insightful experience. It allowed us to create a biometric system that not only interacted with the real world, but also demonstrated the importance of FAR and FRR values. The blaster ultimately led us to choose a higher threshold than what was initially recommended by our data, as we decided to prioritize reducing false positives, even at the cost of increasing false rejects. The final product can be seen in Figure 6.

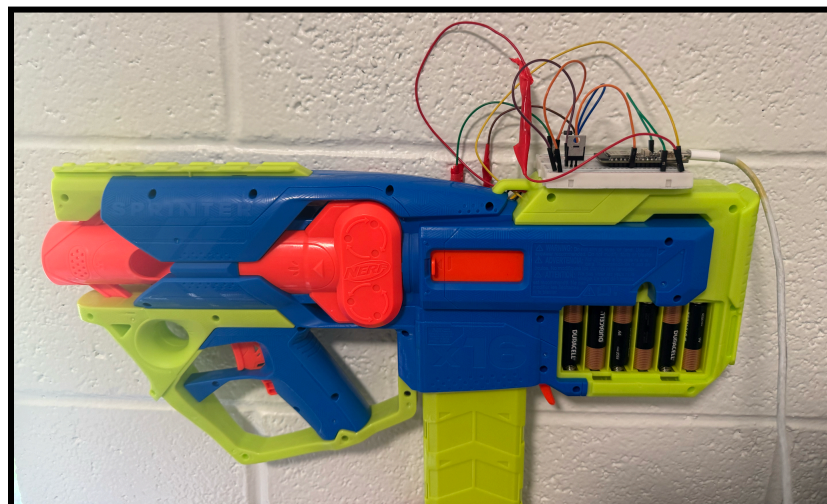


Figure 6: Final Product

Challenges

The first challenge we encountered was determining how to extract features from the audio file. Our initial idea was to use a regular log spectrogram, but through additional research, we learned about the Mel scale and how it could be used to extract more meaningful information from the signal.

The second challenge came from acquiring audio signals from our microphone. Although we were able to get the microphone recording relatively easily, it took a considerable amount of testing to fully understand its capabilities and limitations. To achieve reliable results, the user must be relatively close to the microphone and speak clearly. We think that this could be fixed either with higher quality microphones or additional signal processing steps to reduce background noise.

Improvements

There are many improvements that can be made to this model. First off the models were trained / fine-tuned on only a small subset of the data because the data was so large and preprocessing would take too long along with the training. Model performance could be greatly improved by using all of the data.

Another potential improvement we would like to make would be bluetooth support. Instead of having to be directly connected to the laptop for the blaster activation to work, the blaster would automatically connect to the laptop over bluetooth and the activation signals would be sent over this connection. This is possible without current hardware, since the Feather ESP32-S3 has bluetooth capabilities built into it.

The biggest improvement would be to host all of the computations directly within the blaster. To achieve this we would have to use an edge AI chip to run the models, as well as microphones that can be placed on the blaster. The models would most likely need to be quantized and distilled down to run on one of these edge chips.

Conclusion

In this project, we implemented and evaluated the effectiveness of a speaker recognition biometric system. We observed strong performance in our tests using both our fine-tuned and homegrown transformer models. We then extended this biometric system to a physical application by activating a Nerf blaster motor. However, this biometric approach may not be suitable for high-security applications. During testing, we observed a noticeable number of false positives, and the system is also potentially susceptible to replay attacks. We do

believe, however, that this biometric system could be improved through the use of higher-quality microphones, better handling of background noise, and additional preprocessing steps.

Overall, we learned how to implement a biometric system that was previously unfamiliar to us and how to integrate it into a real-world application. We also learned that signal processing is extremely important in biometrics and plays a key role in achieving good performance. Finally, we were able to apply our prior knowledge to create a new system that has not been widely implemented in this form.